Cimple

Language Documentation

Cimple is a beginner-friendly programming language with a C-like syntax. It is designed to make learning to program easier, without having to worry about complex concepts such as pointers and manual memory management.

1. Data Types

A data type tells the language what kind of value a variable holds.

Keyword	Description	Example
`integer`	Whole number (32-bit)	`integer x = 5;`
`double`	Decimal number (64-bit)	`double pi = 3.14;`
`string`	Text (sequence of characters)	`string name = "Anna";`
`boolean`	True/false value	`boolean ok = true;`
`void`	No return value (functions only)	`void myFunction() {...}`

2. Variables

Variables are declared with a data type, a name, and an initial value:

```
integer age = 17;
string message = "Hello world";
double price = 49.95;
boolean active = true;
```

A variable's value can be reassigned after declaration:

```
age = 18;
```

3. Operators

3.1 Arithmetic

Operator	Description	Example
`+`	Addition	`integer sum = 3 + 2;`
`-`	Subtraction	`integer dif = 5 - 1;`
`*`	Multiplication	`integer prod = 4 * 2;`

Operator	Description	Example
/	Division	double q = 10 / 4;
MOD	Modulus (remainder)	integer r = 10 MOD 3;

3.2 Comparison

Operator	Description
==	Equal to
!=	Not equal to
<	Less than
>	Greater than
<=	Less than or equal to
>=	Greater than or equal to

3.3 Logical Operators

Operator	Description
AND	Logical and
OR	Logical or
!	Negation

4. Control Structures

4.1 If / else

Executes code based on a condition:

```
if(x > 10) {
    print("Large number");
} else {
    print("Small number");
}
```

4.2 While loop

Repeats a block of code as long as the condition is true:

```
integer i = 0;
while(i < 5) {
    print(i);
    i = i + 1;
}
```

5. Arrays

An array stores multiple values of the same type.

Declare with values:

```
integer numbers[] = [1, 2, 3, 4, 5];
```

Declare an empty array with a fixed size:

```
integer numbers[10];
```

Access or set a value:

```
numbers[0] = 99;
print(numbers[0]);
```

> *The first element always has index 0.*

6. Functions

A function is defined with a return type, a name, parameters, and a return expression:

```
integer addTwo(integer a, integer b) {
    return a + b;
}
```

A function with no return value uses void:

```
void sayHello(string name) {
    print("Hello, " + name);
    return 0;
}
```

Call a function like this:

```
integer result = addTwo(3, 4);
sayHello("Maria");
```

7. Print

print is used to display output in the terminal:

```
print("Hello world");
print(x);
print("The result is: ", x);
```

8. Comments

Comments explain code and are ignored by the language:

```
/* This is a comment */

/*
   This is a
   multi-line comment
*/
```

9. A Complete Example

```
integer factorial(integer n) {
    integer result = 1;
    while(n > 1) {
        result = result * n;
        n = n - 1;
    }
    return result;
}

integer answer = factorial(5);
print("5! = ", answer);
```

10. Operator Precedence

Operators are evaluated in this order (high to low):

Precedence	Operator(s)
1 (highest)	( )
2	!
3	*, /, MOD
4	+, -
5	<, <=, >, >=
6	==, !=
7	AND, OR
8 (lowest)	= (assignment)

11. Formal Semantics

Cimple uses big-step operational semantics with an environment-store model.

11.1 Environment-Store Model

Variables are handled through two structures:

- Environment (E): Maps variable names to memory locations.
- Store (σ): Maps memory locations to values.
- State: The combination σ ∘ E, which maps variables directly to their values.

11.2 Expressions

Expressions evaluate to a value — an integer, double, boolean, or string — without changing the store. Examples: arithmetic, comparisons, boolean logic, function calls, and array lookups.

11.3 Statements

Statements may modify the store. They include variable declarations and assignments, array operations, if/else, while loops, and function calls. A declaration of the form T x = e; evaluates e to a value, allocates a new location, and stores the value there.

11.4 Functions

Functions use static scoping. Each function call creates a new environment for its parameters, mapped to fresh locations in the store. The function environment (π) maps function names to their body, return expression, parameters, and the environment at the time of declaration.

11.5 Type System

Cimple performs compile-time type checking to catch errors before the program runs. The type system ensures that: expressions have consistent types, array elements all share the same type, function arguments match declared parameter types, and boolean conditions are used in if/else and while.

11.6 Error Handling

Division by zero results in abnormal termination (ABORT). Type mismatches detected at compile time prevent the program from running. Runtime errors that cannot be caught statically also result in ABORT.

12. Rules to Remember

- All statements end with ;
- Code blocks are enclosed in { }
- true and false are written in lowercase
- The language is case-sensitive — x and X are different variables
- Division by 0 is not allowed and will terminate the program
- A variable must be declared before it is used

- All elements in an array must be of the same type